



# PPQ 4

## Part a

I established a `simple_md.log.ref` which is below:

| PE                            | KE | Error                  |
|-------------------------------|----|------------------------|
| 49854482.95817039161920547485 |    | 0.00000000000000000000 |
| 0.00000000000000000000        |    |                        |
| 49854482.95817039161920547485 |    | 0.00243672236278765072 |
| 0.000000000004887664439       |    |                        |

I then changed optimisation flags for the compiler to get the best reference time (This was done on my Home PC with an Intel i5-10400F CPU processor). I used the Linux `time` command, extracting the wall clock time. The script for this can be found in `run_opt_flags.sh`.

Current OPT\_FLAGS: -00

-----

Execution Time: 0:33.51

-----

Current OPT\_FLAGS: -01

-----

Execution Time: 0:09.33

-----

Current OPT\_FLAGS: -02

-----

Execution Time: 0:01.92

-----

Current OPT\_FLAGS: -02 -ftree-vectorize -funroll-all-loops -mavx512f

-----

Execution Time: 0:01.38

-----

Current OPT\_FLAGS: -03

-----

Execution Time: 0:01.47

-----

Current OPT\_FLAGS: -03 -march=native -funroll-all-loops -flto

-----

Execution Time: 0:01.36

-----

Current OPT\_FLAGS: -03 -funroll-all-loops -flto -mavx2

-----

Execution Time: 0:01.50

-----

I found that for my Home PC, the best flags were

`-O3 -march=native -funroll-all-loops -flto`. This is already ~24x speedup, without any direct code optimisations.

## Part b

### Only profiling

I now switched over to my laptop (AMD Ryzen 5 3500U processor) and established a reference profile, before making any changes to the code, for which the flat profile is below:

Each sample counts as 0.01 seconds.

| %<br>time | cumulative<br>seconds | self<br>seconds | calls      | self<br>s/call | total<br>s/call | name    |
|-----------|-----------------------|-----------------|------------|----------------|-----------------|---------|
| 54.05     | 16.93                 | 16.93           | 3          | 5.64           | 10.43           | compute |
| 21.72     | 23.74                 | 6.80            | 899910000  | 0.00           | 0.00            | dv      |
| 18.54     | 29.55                 | 5.81            | 2099790000 | 0.00           | 0.00            | min     |
| 5.59      | 31.30                 | 1.75            | 299970000  | 0.00           | 0.00            | v       |
| 0.10      | 31.33                 | 0.03            |            |                |                 | _init   |
| 0.00      | 31.33                 | 0.00            | 2          | 0.00           | 0.00            | update  |
| 0.00      | 31.33                 | 0.00            | 1          | 0.00           | 0.00            | init    |

Note some discrepancies that will continue to arise between the flat profile and the `time` command. `gprof` does not profile things like I/O output, so will generally show a faster time than `time`. However, as changes are made, both times go down, thus it is not a huge problem. Also note that both `update()` and `init()` are not profiled at all, even though they both do 30,000 iterations each. This is likely due to the `gprof` sampling frequency at 0.01s.

It is obvious that the hot spot for this code is the `compute()` function, thus I started there. All my major changes were made to that function, and they are described below. The full source code can be found in submitted `Q4_bmjm501_md_stuff.c`.

```

/* CHANGE 4: Added MIN macro, to reduce the function overhead
of calling min() */
/* Define a ternary operation for minimum value */
#define MIN(a, b) ((a) < (b) ? (a) : (b))

/* MORE CODE */

void compute()
{
    int i, j, k;
    double rij[ndim];
    double d, d2;

    PE = 0;
    KE = 0;

    /* For each particle: */
    for (i = 0; i < nparts; i++)
    {
        /* Initialise forces to zero */
        for (k = 0; k < ndim; k++)
        {
            force[i][k] = 0.0;
        }

        /* CHANGE 5 Part 1: Iterate only through the lower diagonal
        to reduce computation */
        for (j = 0; j < i; j++)
        {
            d2 = 0;
            /* CHANGE 1: Fuse the loops over K */
            for (k = 0; k < ndim; k++)
            {
                rij[k] = pos[i][k] - pos[j][k];
                d2 += rij[k] * rij[k];
            }
            d = sqrt(d2);
            /* CHANGE 3: Changed to +=/-= for better memory access*/
            PE += v(d);

```

```

/* CHANGE 2: Introduce temp variable so
only call dv() once per loop, not twice*/
double temp = dv(d) / d;

for (k = 0; k < ndim; k++)
{
    /* CHANGE 6: Introduce temp variable so only reading
    from the array once */
    double force_contribution = rij[k] * temp;
    force[i][k] -= force_contribution;
    /* CHANGE 5 Part 2: Equal and opposite force contribution
    on particle j */
    force[j][k] += force_contribution;
}
}
/* compute kinetic energy */
for (k = 0; k < ndim; k++)
{
    KE += vel[i][k] * vel[i][k];
}
}
KE *= 0.5 * mass;
}

```

A little more detailed changes can be found in `results/changes_made.txt` , or below:

```
Change 1: do we need two for loops? loop fusing
-- no change
Change 2: no need to calculate dv every time,
introduce a temp variable
-- KE changed by (4.3e-19)
Change 3: changed all to += or -=
-- KE changed by (4.3e-19)
Change 4: added MIN macro
-- KE changed by (4.3e-19)
Change 5: implemented N3L to only iterate
over the lower diagonal values
as that matrix is symmetric
-- PE changed by (4e-6), no change in KE
Change 6: introduce another temp variable
-- PE changed by (4e-6), no change in KE
```

The changes produce a significant result on the time `compute()` took, with insignificant differences in results. The changes in the flat profile for `compute()` can be seen below in the first half (focus on the changes in `self.seconds`), and the second half shows the speedup overall (focus on changes in `cumulative.seconds`):

| %<br>time | cumulative<br>seconds | self<br>seconds | calls | self<br>s/call | total<br>s/call | name       |
|-----------|-----------------------|-----------------|-------|----------------|-----------------|------------|
| 50.73     | 13.93                 | 13.93           | 3     | 4.64           | 9.14            | compute    |
| 67.25     | 10.80                 | 10.80           | 3     | 3.60           | 5.35            | compute    |
| 66.75     | 10.34                 | 10.34           | 3     | 3.45           | 5.16            | compute    |
| 73.21     | 10.95                 | 10.95           | 3     | 3.65           | 4.96            | compute*** |
| 77.26     | 6.76                  | 6.76            | 3     | 2.25           | 2.89            | compute    |
| 80.70     | 6.73                  | 6.73            | 3     | 2.24           | 2.77            | compute    |
| -----     |                       |                 |       |                |                 |            |
| 0.00      | 27.46                 | 0.00            | 2     | 0.00           | 0.00            | update     |
| 0.00      | 16.06                 | 0.00            | 1     | 0.00           | 0.00            | init       |
| 0.00      | 15.49                 | 0.00            | 2     | 0.00           | 0.00            | update     |
| 0.00      | 14.95                 | 0.00            | 2     | 0.00           | 0.00            | update***  |
| 0.00      | 8.75                  | 0.00            | 1     | 0.00           | 0.00            | init       |
| 0.00      | 8.34                  | 0.00            | 1     | 0.00           | 0.00            | init       |

\*\*\*While compute() takes longer after this change, the overall time is faster

With each line corresponding to the specific change

The biggest change is **Change 5**. I implemented this change as by Newton's Third Law, the force acting on particle j by particle i is equal and opposite to the force acting on particle i by particle j, and their square distance is the same. Thus only iterating over the lower diagonal of the particle matrix, and updating the force on both particle i and j, we can get dramatic speedup, with insignificant change in the value.

## Compiler Optimisation after profiling

Found also in `./results/opt_flags_log.txt`, we can see the affects on the changes using compiler optimisation as well as the code optimisations:

\*\*\*\*\*ON LAPTOP BEFORE CHANGES\*\*\*\*\*

Current OPT\_FLAGS: -O0

Execution Time: 0:55.22

Current OPT\_FLAGS: -O3 -march=native -funroll-all-loops -flto -mavx2

Execution Time: 0:01.93

\*\*\*\*\*AFTER 3 CHANGES \*\*\*\*\*

Current OPT\_FLAGS: -O0

Execution Time: 0:33.79

Current OPT\_FLAGS: -O3 -march=native -funroll-all-loops -flto -mavx2

Execution Time: 0:01.77

\*\*\*\*\*CHANGE 4 \*\*\*\*\*

Current OPT\_FLAGS: -O0

Execution Time: 0:27.15

Current OPT\_FLAGS: -O3 -march=native -funroll-all-loops -flto -mavx2

Execution Time: 0:01.58

\*\*\*\*\*CHANGE 5\*\*\*\*\*

Current OPT\_FLAGS: -O0

```

-----
Execution Time: 0:16.24
-----

Current OPT_FLAGS: -O3 -march=native -funroll-all-loops -flto -mavx2
-----
Execution Time: 0:01.30
-----

*****CHANGE 6*****

Current OPT_FLAGS: -O0
-----
Execution Time: 0:13.78
-----

Current OPT_FLAGS: -O3 -march=native -funroll-all-loops -flto -mavx2
-----
Execution Time: 0:01.14
-----

```

Overall without any compiler optimisation, from zero changes to six changes there is a speedup of  $\sim 4.2x$ , and with compiler optimisation there is a speedup of  $\sim 1.7x$ . Overall from no changes, no compiler optimisation to all changes and compiler optimisation, there is a  $\sim 48x$  speedup.

Notably on VIKING, the timing results are:

```
*****VIKING*****
```

```
Current OPT_FLAGS: -O0
```

```
-----
```

```
Execution Time: 0:11.83
```

```
-----
```

```
Current OPT_FLAGS: -O3 -march=native -funroll-all-loops -flto -mavx2
```

```
-----
```

```
Execution Time: 0:00.81
```

```
-----
```

This is with all changes implemented. As can be seen, it is possible to get this to less than a second.